

Java Concepts in Detail

A structured guide from platform basics to modern Java, with code examples. A condensed summary is included on the last page.

1. Platform Basics

Component	Role
JDK	Java Development Kit: compiler (javac), tools, and the JRE. Needed to develop.
JRE	Java Runtime Environment: JVM plus core libraries. Needed to run.
JVM	Java Virtual Machine: loads classes, verifies bytecode, executes it (interpreter + JIT).

- Source **.java** is compiled to platform-independent bytecode **.class**; the JVM runs it on any OS ("write once, run anywhere").
- The **JIT compiler** detects hot code paths and compiles them to native machine code at runtime.
- The **class loader** loads classes on demand; the **bytecode verifier** checks safety before execution.

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, Java!");  
    }  
}  
// javac Hello.java    -> Hello.class  
// java Hello          -> Hello, Java!
```

2. Core Syntax and Data Types

Type	Size	Notes
byte	8-bit	-128 to 127
short	16-bit	-32,768 to 32,767
int	32-bit	Default integer type
long	64-bit	Use suffix L , e.g. 10L
float	32-bit	Use suffix f
double	64-bit	Default decimal type
char	16-bit	Unicode character
boolean	JVM-dependent	true / false

- Everything that is not a primitive is a **reference type** (objects, arrays, Strings).
- **Wrapper classes** (**Integer**, **Double**, **Boolean**...) enable autoboxing/unboxing and are required in generics.
- **Control flow**: **if/else**, **switch**, **for**, **while**, **do-while**, enhanced **for-each**, **break**, **continue**.
- **Strings are immutable**. Use **StringBuilder** (not thread-safe, fast) or **StringBuffer** (thread-safe) for repeated modification.

```
String a = "java";
```

```
String b = "java";
String c = new String("java");
System.out.println(a == b);      // true  (same object in the String pool)
System.out.println(a == c);      // false (different objects)
System.out.println(a.equals(c)); // true  (same content)

Integer x = 10;    // autoboxing
int y = x;         // unboxing
```

Rule of thumb: use `==` for primitives, `.equals()` for object content.

3. Object-Oriented Programming

Class and Object

A class is a blueprint; an object is an instance created with `new`. Constructors initialize the object and can be overloaded. Use `this(...)` to chain constructors.

The Four Pillars

- **Encapsulation:** keep fields `private`, expose behavior through public methods (getters/setters).
- **Inheritance:** `extends` reuses parent behavior. Java allows single class inheritance only.
- **Polymorphism:** compile-time via overloading; runtime via overriding (the JVM selects the method by the actual object type).
- **Abstraction:** hide implementation using `abstract` classes and `interfaces`.

```
abstract class Animal {
    protected String name;
    Animal(String name) { this.name = name; }
    abstract String sound(); // must be implemented by subclasses
    void describe() { System.out.println(name + " says " + sound()); }
}

class Dog extends Animal {
    Dog(String name) { super(name); }
    @Override String sound() { return "Woof"; }
}

class Cat extends Animal {
    Cat(String name) { super(name); }
    @Override String sound() { return "Meow"; }
}

Animal a = new Dog("Rex"); // reference type Animal, object type Dog
a.describe();              // runtime polymorphism -> "Rex says Woof"
```

Abstract Class vs Interface

Feature	Abstract class	Interface
State (instance fields)	Yes	No (only <code>public static final</code> constants)
Constructors	Yes	No
Inheritance	A class extends only one	A class can implement many
Methods	Abstract and concrete	Abstract, <code>default</code> , <code>static</code> , <code>private</code> (Java 8/9+)

Feature	Abstract class	Interface
Use when	Sharing code/state among related classes	Defining a capability or contract

Access Modifiers

Modifier	Class	Package	Subclass	World
<code>private</code>	Yes	No	No	No
default (none)	Yes	Yes	No	No
<code>protected</code>	Yes	Yes	Yes	No
<code>public</code>	Yes	Yes	Yes	Yes

- `static`: belongs to the class, not an instance. `final`: variable cannot be reassigned, method cannot be overridden, class cannot be extended.
- `this` refers to the current object; `super` refers to the parent class.
- **Overloading** = same name, different parameters (compile time). **Overriding** = same signature in a subclass (runtime).

4. Exception Handling

- Hierarchy: `Throwable` splits into `Error` (serious JVM problems, e.g. `OutOfMemoryError`) and `Exception`.
- **Checked exceptions** (e.g. `IOException`, `SQLException`) must be caught or declared with `throws`.
- **Unchecked exceptions** (`RuntimeException` subclasses such as `NullPointerException`, `ArrayIndexOutOfBoundsException`) need not be handled.
- `finally` always runs (except on `System.exit` or JVM crash). Prefer **try-with-resources** for anything `AutoCloseable`.

```

class InsufficientFundsException extends Exception {
    InsufficientFundsException(String msg) { super(msg); }
}

void withdraw(double balance, double amount) throws InsufficientFundsException {
    if (amount > balance) throw new InsufficientFundsException("Balance too low");
}

try (BufferedReader br = new BufferedReader(new FileReader("data.txt"))) {
    System.out.println(br.readLine());
} catch (FileNotFoundException e) {
    System.out.println("File missing: " + e.getMessage());
} catch (IOException e) {
    e.printStackTrace();
} finally {
    System.out.println("Done");
}

```

Best practices: catch specific exceptions, never swallow them silently, and do not use exceptions for normal control flow.

5. Collections Framework

Interface	Implementation	Characteristics
List	ArrayList	Resizable array; O(1) get, O(n) middle insert; allows duplicates
List	LinkedList	Doubly linked; O(1) insert/remove at ends; O(n) get
Set	HashSet	No duplicates; unordered; O(1) average operations
Set	LinkedHashSet	Keeps insertion order
Set	TreeSet	Sorted; O(log n); red-black tree
Map	HashMap	Key-value; unordered; allows one null key; O(1) average
Map	LinkedHashMap	Keeps insertion (or access) order
Map	TreeMap	Sorted by key; O(log n)
Queue/Deque	ArrayDeque, PriorityQueue	FIFO/LIFO; priority-ordered (min-heap by default)

- **HashMap internals:** key's `hashCode()` picks a bucket; `equals()` resolves collisions. Buckets become balanced trees when they grow large (Java 8+).
- **Contract:** if two objects are `equals()`, they must have the same `hashCode()`. Override both together.
- **Fail-fast iterators** throw `ConcurrentModificationException` if the collection is modified during iteration; use `Iterator.remove()` instead.
- `Comparable` defines natural ordering; `Comparator` defines custom ordering.

```
List<String> names = new ArrayList<>(List.of("Asha", "Ravi", "Meera"));
names.sort(Comparator.comparing(String::length).thenComparing(Comparator.naturalOrder()));
```

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Math", 90);
marks.merge("Math", 5, Integer::sum); // 95
marks.computeIfAbsent("Physics", k -> 80);
```

```
for (Map.Entry<String, Integer> e : marks.entrySet()) {
    System.out.println(e.getKey() + " -> " + e.getValue());
}
```

6. Generics

- Generics give compile-time type safety and remove casts: `List<String>` instead of raw `List`.
- **Type erasure:** generic type information is removed after compilation, so you cannot do `new T()` or `instanceof List<String>`.
- **Wildcards:** `? extends T` (read-only producer), `? super T` (consumer), `?` (unknown). Remember **PECS: Producer Extends, Consumer Super**.
- Generics work only with reference types (`List<int>` is illegal; use `List<Integer>`).

```
class Box<T extends Comparable<T>> {
    private T value;
    Box(T value) { this.value = value; }
    boolean isBigger(Box<T> other) { return value.compareTo(other.value) > 0; }
}
```

```

static <T> void swap(T[] arr, int i, int j) { // generic method
    T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
}

static double sum(List<? extends Number> nums) { // upper-bounded wildcard
    double total = 0;
    for (Number n : nums) total += n.doubleValue();
    return total;
}

```

7. Multithreading and Concurrency

- A thread is a unit of execution. Create one via **Thread**, **Runnable**, **Callable**, or (preferred) an **ExecutorService** thread pool.
- **Thread lifecycle**: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, TERMINATED.
- **synchronized** gives mutual exclusion and visibility. **volatile** guarantees visibility but not atomicity. **AtomicInteger** gives lock-free atomic updates.
- **Concurrent collections**: **ConcurrentHashMap**, **CopyOnWriteArrayList**, **BlockingQueue**.
- **Common problems**: race conditions, deadlock (circular lock waiting), starvation, livelock.
- **Virtual threads** (Java 21) are lightweight threads that make blocking-style code scale.

```

class Counter {
    private int count = 0;
    synchronized void increment() { count++; }
    synchronized int get() { return count; }
}

ExecutorService pool = Executors.newFixedThreadPool(4);
Counter counter = new Counter();
for (int i = 0; i < 1000; i++) {
    pool.submit(counter::increment);
}
pool.shutdown();
pool.awaitTermination(5, TimeUnit.SECONDS);
System.out.println(counter.get()); // 1000

CompletableFuture<String> f = CompletableFuture
    .supplyAsync(() -> "data")
    .thenApply(String::toUpperCase);
System.out.println(f.join()); // DATA

```

8. Memory Management and Garbage Collection

- **Stack**: one per thread; stores method frames, local variables, and references. Freed automatically when a method returns.
- **Heap**: shared; stores all objects. Divided into **Young** generation (Eden + Survivor spaces) and **Old** generation.
- **Metaspace**: stores class metadata (replaced PermGen in Java 8).
- **Garbage collection**: objects unreachable from GC roots (thread stacks, static fields) are reclaimed. New objects start in Eden; survivors are promoted to Old.
- **Collectors**: Serial, Parallel, **G1** (default in modern Java), **ZGC** and Shenandoah (low pause).
- **Memory leaks** still occur through lingering references: static collections, unclosed resources, unregistered listeners, **ThreadLocal** misuse.

- **Reference types:** strong, `SoftReference`, `WeakReference`, `PhantomReference`.

9. Modern Java (Java 8 and Later)

- **Lambdas and functional interfaces:** `Function`, `Predicate`, `Consumer`, `Supplier`, `BiFunction`.
- **Streams API:** declarative data processing (filter, map, reduce, collect).
- **Optional:** explicit handling of absent values instead of `null`.
- **var** (Java 10), **text blocks** (Java 15), **records** (Java 16), **sealed classes** (Java 17), **switch expressions** and **pattern matching** (Java 14-21).

```
List<Integer> nums = List.of(3, 8, 5, 12, 7);
List<Integer> result = nums.stream()
    .filter(n -> n > 5)
    .map(n -> n * 2)
    .sorted()
    .toList();                                // [14, 16, 24]

Optional<String> name = Optional.of("Java");
System.out.println(name.map(String::toUpperCase).orElse("none"));

record Student(String name, int marks) {}      // immutable data carrier

String grade = switch (new Student("Asha", 82).marks() / 10) {
    case 10, 9 -> "A";
    case 8     -> "B";
    default    -> "C";
};

Object o = "hello";
if (o instanceof String str && str.length() > 3) {    // pattern matching
    System.out.println(str.toUpperCase());
}
```

10. I/O, JDBC, Reflection and Ecosystem

- **File I/O:** classic `java.io` (streams, readers/writers) and modern `java.nio.file` (`Files`, `Path`). **Serialization** converts objects to bytes; mark fields `transient` to skip them.
- **JDBC:** standard API for relational databases (`Connection`, `PreparedStatement`, `ResultSet`). Always use prepared statements to prevent SQL injection.
- **Annotations and reflection:** inspect and manipulate classes at runtime; the basis of frameworks like Spring and Hibernate.
- **Modules** (Java 9+), **build tools** (Maven, Gradle), **testing** (JUnit, Mockito), **frameworks** (Spring Boot, Hibernate/JPA).

```
Path path = Path.of("notes.txt");
Files.writeString(path, "Hello\nJava");
String sql = "SELECT name FROM students WHERE marks > ?";
try (Connection con = DriverManager.getConnection(url, user, pass);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, 60);
    ResultSet rs = ps.executeQuery();
    while (rs.next()) System.out.println(rs.getString("name"));
}
```

Condensed Summary

The same material, reduced to what to remember.

Topic	Key points
Platform	Source to bytecode to JVM (JIT-optimized). JDK = JRE + tools; JRE = JVM + libraries.
Syntax	8 primitives, everything else is a reference. Strings are immutable. <code>==</code> vs <code>.equals()</code> .
OOP	Encapsulation, inheritance (single class), polymorphism (overload = compile time, override = runtime), abstraction (abstract class = state + code, interface = contract).
Exceptions	Checked (must handle) vs unchecked. <code>try/catch/finally</code> , try-with-resources, custom exceptions.
Collections	List (ordered), Set (unique), Map (key-value), Queue. Override <code>equals()</code> and <code>hashCode()</code> together. Comparable vs Comparator.
Generics	Compile-time type safety; type erasure at runtime; PECS for wildcards.
Concurrency	Executors over raw threads; <code>synchronized</code> , <code>volatile</code> , atomics; concurrent collections; avoid deadlock and races.
Memory	Stack = frames, Heap = objects (Young/Old), Metaspace = class data. GC reclaims unreachable objects; leaks come from lingering references.
Modern Java	Lambdas, streams, Optional, <code>var</code> , records, sealed classes, switch expressions, pattern matching, virtual threads.
Ecosystem	<code>java.nio.file</code> , JDBC with PreparedStatement, reflection/annotations, Maven/Gradle, Spring Boot.